



@ coding.artist

COMPLETE

JS

JAVASCRIPT NOTES

Your ultimate JS guide! ❤️



Concepts



Examples

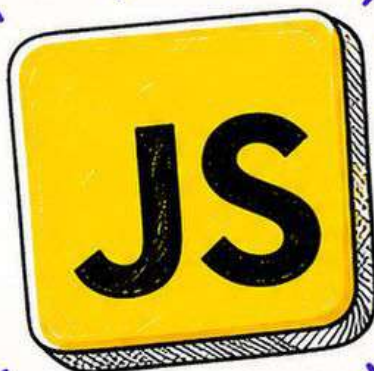


Cheat Sheets



Best Practices

```
// Hello, JavaScript! 🙌  
const msg = "Keep Coding";  
let skills = ["HTML", "CSS",  
             "JS"];  
  
function learn() {  
  console.log(msg);  
}  
  
learn();  
// You got this! 💪
```



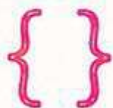
EVERYTHING YOU NEED!

- ✓ Beginner to Advanced
- ✓ Examples with Output
- ✓ Handwritten Notes
- ✓ Quick Reference Tables
- ✓ Interview Essentials

Easy to learn!



BASICS



CONCEPTS



PRACTICAL



PROJECTS



SUCCESS

Let's
make magic
with code! ✨

```
if (you.keepCoding) {  
  success = "Guaranteed!";  
} else {  
  success = "Keep Going!";  
}
```

Code today,
Change tomorrow.

15

PAGES OF
POWERFUL
JAVASCRIPT
NOTES



codingartistweb.com/shop

JAVASCRIPT



JavaScript is a lightweight, interpreted, or just-in-time compiled programming language with first-class functions.



1 WHAT IS JAVASCRIPT?

- ✓ JS is the language of the web.
- ✓ It makes web pages interactive and dynamic.
- ✓ Works in all modern browsers without installation.
- ✓ Used for Frontend, Backend (Node.js), Mobile, Desktop, Games and more!



Web



Backend



Mobile



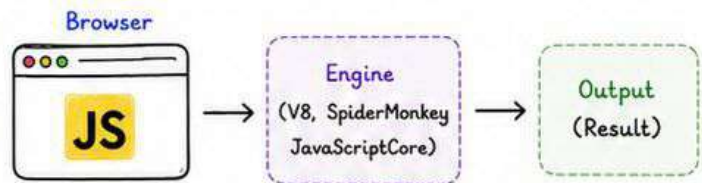
Desktop



Games

2 WHERE DOES JS RUN?

JavaScript runs in the browser using the engine built into it.



- ★ Popular JS Engines: V8 (Chrome, Edge, Node.js)
SpiderMonkey (Firefox) | JavaScriptCore (Safari)

3 SIMPLE EXAMPLE



```
// This is a comment  
let message = "Hello, JavaScript!";  
console.log(message); // Hello, JavaScript!
```

Run this in your browser console to see the output.



4 HOW TO ADD JAVASCRIPT TO HTML?

- Inline JS
- Internal JS
- External JS

JS can be added anywhere in <html> using <script> tag.



5 BASIC SYNTAX RULES

✓ VALID

- ✓ Case sensitive (myVar and myvar are different)
- ✓ Statements end with a semicolon ;
- ✓ Whitespace (spaces, tabs, new lines) are ignored (except in strings)
- ✓ Comments help to explain the code

```
let name = "Coding Artist";  
const age = 30;  
var isLearning = true;  
console.log(name);  
// This is a single line comment  
/*  
This is a multi-line comment  
*/
```

✗ INVALID

```
let name = "Coding"      ✗ missing ;  
const age = 30           ✗ missing ;  
Var isLearning = true;   ✗ 'Var' (use var)  
console.Log(name);       ✗ 'L' should be lowercase  
// This is a comment */ ✗ wrong comment
```

6 HELLO WORLD!

The traditional first program in JS 😊



```
console.log("Hello, World!");
```

Opens console (F12 or Ctrl+Shift+I) to see the output.



7 JAVASCRIPT IS...



Dynamic
Types are checked at runtime



Flexible
Supports multiple programming styles



Cross-platform
Runs in browsers, servers, devices



Powerful
Small code, big impact!



Remember: Practice a little every day and JS will become your superpower! 💪

VARIABLES, DATA TYPES & TYPE CONVERSION

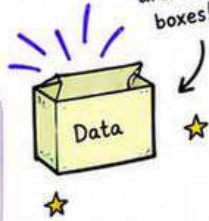


Variables store data. Data types tell us what kind of data it is. Type conversion changes the data from one type to another.

1 WHAT IS A VARIABLE?

- ✓ A variable is a container for storing data.
- ✓ Declared using `let`, `const` or `var` (`var` is old).
- ✓ Can hold different values over time.
- ✓ Names are case-sensitive.

Variables are like boxes!



EXAMPLE

```
let name = "Alice"; // string
const age = 25; // number
var isStudent = true; // boolean
```

2 THE 6 JAVASCRIPT DATA TYPES

Type	Description	Example
String	Text data	"Hello"
Number	Numeric values	42, 3.14
Boolean	True or false	true, false
Undefined	Variable declared but not assigned	let x; // undefined
Null	Intentional absence of value	null
Object	Complex data (key-value pairs, arrays, etc.)	{name: "John"}

6 types + 1 special one!

3 TYPE CHECKING

Use the `typeof` operator to find the data type of a value.

EXAMPLE

```
console.log(typeof "Hello"); // string
console.log(typeof 42); // number
console.log(typeof true); // boolean
console.log(typeof undefined); // undefined
console.log(typeof null); // object (quirk!)
```

`typeof` is your friend!

4 CONVERTING TO A STRING

Use `.toString()` or `String()`.

EXAMPLE

```
let num = 100;
console.log(num.toString()); // "100"
console.log(String(num)); // "100"
```

Great for adding or displaying data!

5 CONVERTING TO A NUMBER

Use `Number()` or `parseInt()/parseFloat()`.

EXAMPLE

```
let str = "42";
console.log(Number(str)); // 42
console.log(parseInt(str)); // 42
console.log(parseFloat("3.14")); // 3.14
```

NaN means "Not a Number"



6 CONVERTING TO A BOOLEAN

Use `Boolean()`.

EXAMPLE

```
console.log(Boolean(1)); // true
console.log(Boolean(0)); // false
console.log(Boolean("")); // false
console.log(Boolean("Hello")); // true
```

Empty values are falsy!



7 TYPE COERCION (IMPLICIT)

JavaScript automatically converts types in some cases.

EXAMPLE

```
console.log("5" + 2); // "52" (string)
console.log("5" - 2); // 3 (number)
console.log(true + 1); // 2 (number)
```

Be careful! It can be surprising!

8 TYPE CONVERSION SUMMARY

Goal	Method
String	<code>String(value)</code> , <code>.toString()</code>
Number	<code>Number(value)</code> , <code>parseInt()</code> , <code>parseFloat()</code>
Boolean	<code>Boolean(value)</code>
Check type	<code>typeof value</code>
Explicit conversion	Use the above methods



QUICK TIPS:

- ✓ Use `const` for values that don't change.
- ✓ Use `let` for values that will change.
- ✓ Always check types when debugging.
- ✓ Avoid unnecessary type coercion.

Right type, right result!

OPERATORS & EXPRESSIONS

JS

Operators perform operations on values and variables.

1 WHAT IS AN OPERATOR?

- ✓ Operators are special symbols that perform operations on one, two or three operands (values).
- ✓ They return a result.

5 + 3 // 8

x > 10 // true

name + "!" // "Coding Artist!"



2 TYPES OF OPERATORS

- ✓ Arithmetic Operators
- ✓ Assignment Operators
- ✓ Comparison Operators
- ✓ Logical Operators
- ✓ String Operators
- ✓ Ternary (Conditional) Operator
- ✓ Bitwise Operators
- ✓ Type Operators
- ✓ Other Operators

OPERATORS MAKE
YOUR CODE:

- ★ Shorter
- ★ Smarter
- ★ More Powerful

3 ARITHMETIC OPERATORS

Operator	Name	Description	Example	Result
+	Addition	Adds two values	5 + 3	8
-	Subtraction	Subtracts right value from left	5 - 3	2
*	Multiplication	Multiplies two values	5 * 3	15
/	Division	Divides left value by right	15 / 3	5
%	Modulus (Remainder)	Returns remainder	10 % 3	1
**	Exponentiation	Left value to the power of right	2 ** 3	8
++	Increment	Increases value by 1	let x = 5; x++;	6
--	Decrement	Decreases value by 1	let x = 5; x--;	4

4 ASSIGNMENT OPERATORS

Operator	Example	Same As
=	x = 10	x = 10
+=	x += 5	x = x + 5
-=	x -= 3	x = x - 3
*=	x *= 2	x = x * 2
/=	x /= 4	x = x / 4
%=	x %= 2	x = x % 2
**=	x **= 3	x = x ** 3

5 COMPARISON OPERATORS

Operator	Meaning	Example	Result
==	Equal to	5 == "5"	true
===	Strict equal to	5 === "5"	false
!=	Not equal to	5 != 3	true
!==	Strict not equal to	5 !== "5"	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater than or equal	5 >= 5	true
<=	Less than or equal	5 <= 5	true

6 LOGICAL OPERATORS

Operator	Meaning	Example	Result
&&	AND	(5 > 3) && (2 < 4)	true
	OR	(5 < 3) (2 < 4)	true
!	NOT	!(5 > 3)	false



AND (&&) → true if both conditions are true

OR (||) → true if at least one condition is true

NOT (!) → reverses the value

7 STRING OPERATORS

Operator	Meaning	Example	Result
+	Concatenation	"Hello" + " JS"	"Hello JS"
+=	Concatenate & assign	let s = "Hi"; s += " there";	"Hi there"

+ can also work as addition if both operands are numbers.



8 TERNARY OPERATOR

Short hand for if...else

condition ? expressionIfTrue : expressionIfFalse

let age = 18;

let result = age >= 18 ? "Adult" : "Minor";

↑
condition↑
if true↑
if false

9 TYPE OPERATORS

Operator	Meaning	Example
typeof	Returns type of a value	typeof 123 // "number" typeof "Hi" // "string"
instanceof	Checks if object is instance of a constructor	[] instanceof Array // true



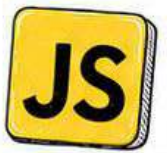
TIP: Use parentheses () to make expressions clear and avoid confusion.

</> Coding Artist </>

codingartistweb.com/shop



CONDITIONAL STATEMENTS & TERNARY OPERATOR



Make decisions in your code! ❤️

1 WHAT ARE CONDITIONAL STATEMENTS?

- Used to execute different code blocks based on different conditions.
- They help your program make decisions.

```
if (condition) {
  // code if condition is true
} else {
  // code if condition is false
}
```

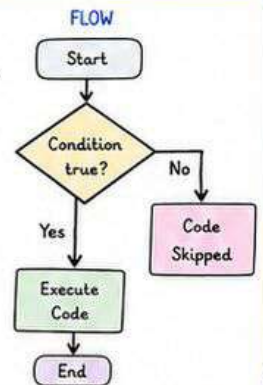
CONDITION
is any expression
that returns
true or false. ❤️

2 IF STATEMENT

Executes a block of code if the condition is true.

```
let age = 18;

if (age >= 18) {
  console.log("You are an adult.");
}
```

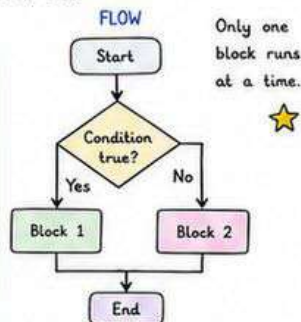


3 IF...ELSE STATEMENT

Executes one block of code if condition is true, otherwise executes another block.

```
let score = 75;

if (score >= 50) {
  console.log("You passed! 🎉");
} else {
  console.log("You failed! 😞");
}
```



4 IF...ELSE IF...ELSE STATEMENT

Use when you have multiple conditions to check.

```
let marks = 82;

if (marks >= 90) {
  console.log("Grade: A");
} else if (marks >= 75) {
  console.log("Grade: B");
} else if (marks >= 50) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
```

CHECKED FROM TOP TO BOTTOM.
First true condition executes. ★

5 NESTED IF STATEMENT

You can use if statements inside another if.

```
let age = 20;

if (age >= 18) {
  if (age >= 21) {
    console.log("You can drink. 🍷");
  } else {
    console.log("You are adult but cannot drink.");
  }
} else {
  console.log("You are underage.");
}
```

TIP
Avoid too much nesting.
It makes code hard to read. ❤️

6 TERNARY OPERATOR (? :)

A short way to write if...else in a single line.

```
condition ? exprIfTrue : exprIfFalse;
```

↑
If condition is true

↑
If condition is false

EXAMPLES

```

• let age = 18;
  let result = (age >= 18) ? "Adult" : "Minor"; // Adult

• let isLoggedIn = true;
  let message = isLoggedIn ? "Welcome!" : "Please login!"; // Welcome!

• let num = 7;
  let type = (num % 2 === 0) ? "Even" : "Odd"; // Odd
  
```

Why use it?

- ✓ Shorter
- ✓ Cleaner
- ✓ Perfect for simple checks

7 QUICK EXAMPLES

Positive or Negative

```
let n = -4;

if (n >= 0) {
  console.log("Positive");
} else {
  console.log("Negative");
}
```

A // Negative

Largest of Two Numbers

```
let a = 12, b = 25;

if (a > b) {
  console.log(a + " is larger");
} else {
  console.log(b + " is larger");
}
```

B // 25 is larger

Leap Year Check

```
let year = 2024;

if ((year % 4 === 0 && year % 100 !== 0) || (year % 400 === 0)) {
  console.log("Leap Year");
} else {
  console.log("Not a Leap Year");
}
```

C // Leap Year

Using Ternary

```
let num = 10;

let result = (num > 0) ? "Positive" : (num < 0 ? "Negative" : "Zero");
```

D // Positive

Small decisions make big differences in code! ❤️

💡 Practice these daily and you'll write smart, dynamic and real-world JS! 😊

</> Coding Artist </>

codingartistweb.com/shop

LOOPS & ITERATION



Repeat tasks, save time, write less code! ❤️

1 WHAT IS A LOOP?

A loop is used to run a block of code multiple times until a condition is met.

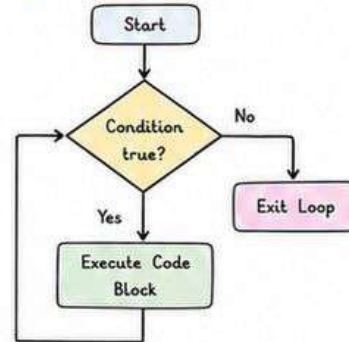
```
// Print numbers from 1 to 5
for (let i = 1; i <= 5; i++) {
  console.log(i);
}

// Output: 1 2 3 4 5
```

Loops help in

- ✓ Repeating tasks
- ✓ Iterating arrays
- ✓ Processing data

2 HOW A LOOP WORKS?



- 1. Initialization**
Start the loop (set counter, variable, etc.)
- 2. Condition Check**
Check if the condition is true.
- 3. Execute Code**
Run the code inside the loop.
- 4. Update**
Update the counter / variable.
- 5. Repeat**
Again check the condition.

3 FOR LOOP

Best when you know how many times you want to loop.

```
for (initialization; condition; update) {
  // code to be executed
}
```

Start counter Check condition Update counter

- ★ `i++` → Increase by 1
- ★ `i--` → Decrease by 1
- ★ `i += 2` → Increase by 2
- ★ `i -= 2` → Decrease by 2

EXAMPLE

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}

// Output: 1 2 3 4 5
```

BREAK & CONTINUE

break → Exit the loop immediately.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) break;
  console.log(i);
}

// Output: 1 2
```

continue → Skip this iteration.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i);
}

// Output: 1 2 4 5
```

4 WHILE LOOP

Use when you don't know how many times the loop will run.

EXAMPLE

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}

// Output: 1 2 3 4 5
```



Don't forget to update the counter, or the loop will run forever! ❤️

When to use?

- ★ `for` → Known count
- ★ `while` → Unknown count
- ★ `do...while` → Run at least once

5 DO...WHILE LOOP

The code runs at least once, then the condition is checked.

```
initialization;
do {
  // code
  // update counter
} while (condition);
```

EXAMPLE

```
let i = 1;
do {
  console.log(i);
  i++;
} while (i <= 5);

// Output: 1 2 3 4 5
```

6 FOR...IN LOOP

Used to loop through the keys of an object.

```
for (key in object) {
  // code
}
```

Use key to access the properties.

EXAMPLE

```
const person = { name: "Mitali",
  age: 30, city: "Pune" };

for (let key in person) {
  console.log(key + ": " + person[key]);
}

// Output:
// name: Mitali
// age: 30
// city: Pune
```

7 FOR...OF LOOP

Used to loop through values of iterable objects like arrays, strings, maps, sets.

```
for (value of iterable) {
  // code
}
```

EXAMPLE

```
const fruits = ["Apple", "Banana", "Mango"];

for (let fruit of fruits) {
  console.log(fruit);
}

// Output:
// Apple
// Banana
// Mango
```



8 INFINITE LOOP

Happens when the condition never becomes false.

```
while (true) {
  console.log("This will run forever!");
}
```



Be careful! Infinite loops can crash your browser.



TIP: Use loops smartly to make your code shorter, cleaner and more powerful! 💡

</> Coding Artist </>

codingartistweb.com/shop

FUNCTIONS & SCOPE



Organize code, reuse logic, understand visibility! ❤️

1 WHAT IS A FUNCTION?

- ✓ A function is a block of code that performs a specific task.
- ✓ Helps in reusability and clean code.

```
function greet(name) {
  console.log("Hello, " + name + "!");
}

greet("Coding Artist"); // Hello, Coding Artist!
```

Think of functions as mini-programs inside your program!

2 FUNCTION SYNTAX

Function Declaration

```
function add(a, b) {
  return a + b;
}

let sum = add(5, 3);
console.log(sum); // 8
```

Function Expression

```
const multiply = function(a, b) {
  return a * b;
};

let result = multiply(4, 3);
console.log(result); // 12
```

★ Function Declaration is hoisted.
Function Expression is not hoisted.



3 TYPES OF FUNCTIONS

1. Named Function

```
function sayHi() {
  console.log("Hi there!");
}

sayHi();
```

★ Has a name. Can be reused anywhere.

2. Anonymous Function

```
const sayHello = function() {
  console.log("Hello!");
};

sayHello();
```

★ No name. Stored in a variable.

3. Arrow Function (ES6)

```
const square = (n) => n * n;

console.log(square(5)); // 25
```

★ Shorter syntax. ★ Does not have its own this.

4 FUNCTION PARAMETERS

Parameters (Inputs)

```
function greet(name) {
  console.log("Hi, " + name);
}

greet("Mitali"); // Hi, Mitali
```

Default Parameters (ES6)

```
function greet(name = "Guest") {
  console.log("Hi, " + name);
}

greet(); // Hi, Guest
greet("John"); // Hi, John
```

Return Statement

```
function add(a, b) {
  return a + b; // returns value back
}

let total = add(10, 20);
console.log(total); // 30
```

return ends the function and sends a value back to where it was called.

5 SCOPE IN JAVASCRIPT

Scope determines the visibility (accessibility) of variables.

1. Global Scope

```
let globalVar = "I am global";

function show() {
  console.log(globalVar); // Accessible
}

show(); // I am global
console.log(globalVar); // I am global
```

★ Declared outside any function. Accessible everywhere.

2. Function Scope

```
function test() {
  let localVar = "I am local";
  console.log(localVar); // Accessible
}

test();
// console.log(localVar);
// ✗ Error: localVar is not defined
```

★ Declared inside a function. Accessible only within that function.

3. Block Scope (let & const)

```
if (true) {
  let a = 10;
  const b = 20;
  console.log(a, b); // 10 20
}

// console.log(a); // ✗ Error
// console.log(b); // ✗ Error
```

★ let and const are block scoped. Works inside { } (blocks).

6 SCOPE CHAIN (LEXICAL SCOPE)

When a variable is not found in the current scope, JS looks in the outer scope and so on.

```
let x = "Global";
function outer() {
  let y = "Outer";
  function inner() {
    let z = "Inner";
    console.log(x); // Global
    console.log(y); // Outer
    console.log(z); // Inner
  }
  inner();
}
outer();
```



JS looks upward in the chain of scopes to find variables.

7 IMPORTANT NOTES

- ✓ Use functions to avoid code repetition.
- ✓ Use meaningful names for functions.
- ✓ Use return to send values back.
- ✓ Use let and const to control scope and avoid bugs.



8 THIS KEYWORD (QUICK NOTE)

this refers to the object that is executing the current function.

```
const user = {
  name: "Mitali",
  sayHi: function() {
    console.log("Hi, " + this.name);
  }
};

user.sayHi(); // Hi, Mitali
```

We'll learn more about this in advanced concepts!



TIP: Master functions and scope, and you'll master clean and powerful JavaScript! 🧡

</> Coding Artist </>

codingartistweb.com/shop

ARRAYS & ARRAY METHODS

Store, organize and work with list of data! ❤️

JS

1 WHAT IS AN ARRAY?

- ✓ An array is a special variable that can hold multiple values in a single variable.
- ✓ Values are stored in order (indexed).

```
let fruits = ["Apple", "Banana", "Mango"];
```

0	1	2	
"Apple"	"Banana"	"Mango"	← Index
			← Values

★ Indexes start at 0.

WHY USE ARRAYS?

- ✓ Store multiple items
- ✓ Access items by index
- ✓ Easy to loop through
- ✓ Many useful methods

2 ACCESSING ARRAY ELEMENTS

```
let colors = ["Red", "Green", "Blue"];
```

```
colors[0]; // "Red"
colors[1]; // "Green"
colors[2]; // "Blue"
colors.length; // 3
```

Useful:

length → returns number of elements

Last index = length - 1

```
colors[colors.length - 1];
// "Blue"
```

3 ADDING & REMOVING ELEMENTS

ADD ELEMENTS

1. push() → adds to end

```
let arr = [1, 2];
arr.push(3);
// [1, 2, 3]
```



REMOVE ELEMENTS

1. pop() → removes from end

```
arr.pop();
// [0, 1, 2]
```

2. shift() → removes from start

```
arr.shift();
// [1, 2]
```

2. unshift() → splice()

```
arr.unshift(0);
// [0, 1, 2, 3]
```

```
arr.splice(1, 1); // remove 1 item at index 1
// [1]
```



4 ARRAY METHODS (COMMONLY USED)

Method	What it does	Example	Result
push()	Add element(s) at end	arr.push(4)	[..., 4]
pop()	Remove last element	arr.pop()	removes last
shift()	Remove first element	arr.shift()	removes first
unshift()	Add element(s) at start	arr.unshift(0)	[0, ...]
splice(index, deleteCount, items...)	Add/Remove elements	arr.splice(1, 2, 'A')	modifies array
slice(start, end)	Returns a portion	arr.slice(1, 3)	new array
indexOf(item)	First index of item	arr.indexOf('A')	index or -1
includes(item)	Check if item exists	arr.includes('A')	true / false
join(separator)	Convert to string	arr.join('-')	"A-B-C"
reverse()	Reverse the array	arr.reverse()	reversed array
sort()	Sort the array	arr.sort()	sorted array

5 ITERATING OVER ARRAYS

1. for Loop

```
let nums = [10, 20, 30];
for (let i = 0; i < nums.length; i++) {
  console.log(nums[i]);
}
// 10 20 30
```



2. for...of Loop (ES6)

```
for (let num of nums) {
  console.log(num);
}
// 10 20 30
```



3. forEach() Method

```
nums.forEach(function(num, index) {
  console.log(index, num);
});
// 0 10 1 20 2 30
```



6 MULTIDIMENSIONAL ARRAYS

An array inside another array.

```
let students = [
  ["Mitali", 90],
  ["John", 85],
  ["Sara", 95]
];

students[0][0]; // "Mitali"
students[1][1]; // 85
```

Index →

	0	1
0	"Mitali"	90
1	"John"	85
2	"Sara"	95

NOTE:

You can think of it as a table (rows and columns).



7 ARRAY DESTRUCTURING (ES6)

Extract values from arrays into variables.

```
let [a, b, c] = [100, 200, 300];
console.log(a); // 100
console.log(b); // 200
console.log(c); // 300
```

Super useful!

8 SPREAD OPERATOR (...)

Create a new array or copy/merge arrays.

```
let arr1 = [1, 2, 3];
let arr2 = [...arr1, 4, 5];
console.log(arr2); // [1, 2, 3, 4, 5]

let copy = [...arr1]; // shallow copy
```



9 QUICK EXAMPLES

```
let letters = ['A', 'B'];
letters.push('C'); // ['A', 'B', 'C']
letters.unshift('Start'); // ['Start', 'A', 'B', 'C']
letters.pop(); // ['Start', 'A', 'B']
letters.shift(); // ['A', 'B']
letters.includes('B'); // true
letters.indexOf('A'); // 0
```



TIP:

Arrays are powerful! Master the methods and practice daily to write cleaner and efficient code.



</> Coding Artist </>

codingartistweb.com/shop

OBJECTS & OBJECT METHODS

Objects store key-value pairs. Powerful, flexible and everywhere! ❤️



1 WHAT IS AN OBJECT?

- ✓ An object is a collection of key-value pairs.
- ✓ Keys (properties) are strings (or Symbols).
- ✓ Values can be any data type.

```
const person = {
  name: "Mitali",
  age: 30,
  isDeveloper: true
};
```

WHY USE OBJECTS?

- ✓ Organized data
- ✓ Easy to access
- ✓ Real-world modeling
- ✓ Many useful methods

2 CREATING OBJECTS

1. Object Literal (Most common)

```
const user = {
  name: "John",
  age: 25,
  email: "john@email.com"
};
```

2. Using new Object()

```
const user = new Object();
user.name = "John";
user.age = 25;
user.email = "john@email.com";
```

3. Using Constructor Function

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
```

```
const p1 = new Person("Mitali", 30);
// p1 -> { name: "Mitali", age: 30 }
```

3 ACCESSING PROPERTIES

Dot Notation

```
const person = { name: "Sara", age: 28 };
console.log(person.name); // "Sara"
console.log(person.age); // 28
```

Bracket Notation

```
console.log(person["name"]); // "Sara"
const key = "age";
console.log(person[key]); // 28
```



NOTE: Use dot notation when key is known.
Use bracket notation when key is variable or has special characters.

4 ADDING, UPDATING & DELETING PROPERTIES

Add Property

```
person.city = "Pune";
// { ..., city: "Pune" }
```

Update Property

```
person.age = 29;
// age updated
```

Delete Property

```
delete person.email;
// email removed
```



NOTE: Properties can be added, updated or deleted anytime.
Objects are mutable! 😊

5 OBJECT METHODS (BUILT-IN)

Method	What it does	Example	Result
Object.keys(obj)	Returns array of keys	Object.keys(person)	["name", "age", "isDeveloper"]
Object.values(obj)	Returns array of values	Object.values(person)	["Mitali", 30, true]
Object.entries(obj)	Returns array of [key, value]	Object.entries(person)	[["name", "Mitali"], ["age", 30], ["isDeveloper", true]]
Object.assign(target, ...)	Copies properties to target	Object.assign({}, person)	Copies all properties
Object.freeze(obj)	Prevents changes	Object.freeze(person)	Cannot add/update/delete
Object.seal(obj)	Prevents add/delete Allows update	Object.seal(person)	Add/Delete ✗ Update ✓

6 LOOPING THROUGH OBJECTS

1. for...in Loop (keys)

```
for (let key in person) {
  console.log(key + ": ", person[key]);
}
// name: Sara
// age: 28
```

2. Object.keys() + forEach

```
Object.keys(person).forEach(key => {
  console.log(key, ": ", person[key]);
});
```



NOTE: for...in → loops through enumerable keys.
Does not guarantee order.

7 NESTED OBJECTS

```
const student = {
  id: 101,
  name: "Mitali",
  address: {
    city: "Pune",
    pin: 411001
  }
};
```

Accessing:

```
student.address.city; // "Pune"
student["address"]["pin"]; // 411001

const { address: { city } } = student;
console.log(city); // "Pune"
```



8 OBJECT DESTRUCTURING (ES6)

1. Basic Destructuring

```
const { name, age } = person;
console.log(name); // "Sara"
console.log(age); // 28
```

2. With Default Values

```
const { name, email = "N/A" } = person;
console.log(email); // "N/A"
```

Rename While Destructuring

```
const { name: fullName, age: userAge } = person;
console.log(fullName); // "Sara"
console.log(userAge); // 28
```

Destructuring makes your code clean & readable! ❤️

9 THIS KEYWORD IN OBJECTS

```
const person = {
  name: "Mitali",
  age: 30,
  greet: function() {
    console.log("Hi, I am " + this.name);
  }
};
```



this refers to the object that is calling the method.
person.greet();
// Hi, I am Mitali

10 QUICK EXAMPLES

1. Check if property exists

```
if ("age" in person) {
  console.log("Age exists!");
}
```

2. Clone an Object

```
const clone = { ...person };
console.log(clone); // shallow copy
```



TIP: Objects are the backbone of real-world JavaScript. Practice daily and master them! 🧡

</> Coding Artist </>

codingartistweb.com/shop

STRINGS, NUMBERS & BUILT-IN METHODS



Powerful methods to write less code and do more!

1 STRINGS IN JAVASCRIPT

- ✓ Strings are used to store text.
- ✓ Use single quotes `'`, double quotes `"` or backticks ```.

```
let name1 = 'Coding Artist';
let name2 = 'JavaScript';
let msg = `Hello, ${name1}!`; // Template literal

console.log(msg); // Hello, Coding Artist!
```

STRING TIPS

- ✓ Immutable (cannot change individual characters)
- ✓ Index starts at 0
- ✓ case-sensitive

2 STRING METHODS

Method	What it does	Example	Result
length	Returns length of string	'Hello'.length	5
charAt(index)	Character at index	'Hello'.charAt(1)	'e'
at(index)	Character at index (ES2022)	'Hello'.at(-1)	'o'
toUpperCase()	Convert to UPPERCASE	'hello'.toUpperCase()	'HELLO'
toLowerCase()	Convert to lowercase	'HELLO'.toLowerCase()	'hello'
trim()	Remove spaces from both ends	' hi '.trim()	'hi'
includes(search)	Check if contains	'hello'.includes('ell')	true
indexOf(search)	First index of search	'hello'.indexOf('l')	2
lastIndexOf(search)	Last index of search	'hello'.lastIndexOf('l')	3
slice(start, end)	Extract part of string	'hello'.slice(1, 4)	'ell'
substring(start, end)	Similar to slice (no negative index)	'hello'.substring(1, 4)	'ell'
replace(old, new)	Replace first match	'hi hi'.replace('hi', 'Hello')	'Hello hi'
replaceAll(old, new)	Replace all matches	'hi hi'.replaceAll('hi', 'Hello')	'Hello Hello'
split(separator)	Split into an array	'a,b,c'.split(',')	['a','b','c']

3 EXAMPLES - STRING METHODS

```
let text = 'JavaScript is Awesome!';
console.log(text.length); // 23
console.log(text.trim()); // 'JavaScript is Awesome!'
console.log(text.toUpperCase()); // 'JAVASCRIPT IS AWESOME!'
console.log(text.includes('Script')); // true
console.log(text.indexOf('is')); // 11
console.log(text.slice(2, 11)); // 'vaScript'
console.log(text.replace('Awesome', 'Great')); // 'JavaScript is Great!'
```

4 NUMBERS IN JAVASCRIPT

- ✓ All numbers are stored as 64-bit floating point.
- ✓ Supports integers, decimals, positive & negative numbers.

```
let a = 10;
let b = 3.14;
let c = -5;
console.log(typeof a); // 'number'
```

NUMBER FACTS

No separate types for int or float.
All numbers are of type 'number'.

5 NUMBER METHODS & PROPERTIES

Method/Property	What it does	Example	Result
toFixed(digits)	Fixed decimal places	(12.345).toFixed(2)	'12.35'
toPrecision(digits)	Precision to total digits	(12.345).toPrecision(3)	'12.3'
toString()	Convert number to string	(123).toString()	'123'
Number()	Convert to number	Number('123')	123
parseInt()	Parse integer	parseInt('123px')	123
parseFloat()	Parse float	parseFloat('12.34px')	12.34
isNaN(value)	Check if Not-a-Number	isNaN('hello')	true
isFinite(value)	Check if finite number	isFinite(10/0)	false

6 EXAMPLES - NUMBER METHODS

```
let num = 123.4567;
console.log(num.toFixed(2)); // '123.46'
console.log(num.toPrecision(4)); // '123.5'
console.log(Number('456.78')); // 456.78
console.log(parseInt('100px')); // 100
console.log(parseFloat('10.5kg')); // 10.5
console.log(isNaN('abc')); // true
console.log(isFinite(999)); // true
```

7 USEFUL BUILT-IN METHODS (GLOBAL)

Method	What it does	Example	Result
Math.round(x)	Round to nearest integer	Math.round(4.6)	5
Math.floor(x)	Round down	Math.floor(4.9)	4
Math.ceil(x)	Round up	Math.ceil(4.1)	5
Math.abs(x)	Absolute value	Math.abs(-7)	7
Math.max(a, b, ...)	Largest value	Math.max(1, 5, 3)	5
Math.min(a, b, ...)	Smallest value	Math.min(1, 5, 3)	1
Math.random()	Random between 0 (incl.) and 1 (excl.)	Math.random()	0 <= x < 1

Method	What it does	Example	Result
Math.pow(x, y)	x to the power y	Math.pow(2, 3)	8
Math.sqrt(x)	Square root	Math.sqrt(16)	4
Math.PI	Value of PI	Math.PI	3.14159...
Date()	Current date & time	new Date()	Date object
Date.now()	Milliseconds since Jan 1, 1970	Date.now()	168... (number)
JSON.stringify(obj)	Convert object to JSON	JSON.stringify({a:1})	'{"a":1}'
JSON.parse(str)	Convert JSON to object	JSON.parse('{"a":1}')	{a: 1}

8 QUICK EXAMPLES

```
// Strings
let str = 'Hello World!';
console.log(str.toUpperCase());
// HELLO WORLD!
console.log(str.slice(0,5));
// Hello
```

```
// Numbers
let x = 9.876;
console.log(x.toFixed(1));
// "9.9"
console.log(Math.sqrt(81));
// 9
```

```
// Random number between 1 and 10
let r = Math.floor(Math.random()*10) + 1;
console.log(r);
// 1 to 10
```

```
// Convert string number to number
let s = '250';
console.log(Number(s) + 50);
// 300
```

REMEMBER

- ✓ Use the right method to save time
- ✓ Explore more methods in MDN Docs

TIP: Practice small examples daily. The more you use these methods, the more natural JavaScript will feel!

</> Coding Artist </>

codingartistweb.com/shop

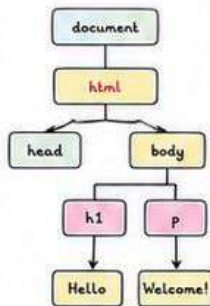
DOM MANIPULATION

Interact with HTML. Update content. Create dynamic web pages!

JS

1 WHAT IS DOM?

- DOM (Document Object Model) represents the structure of an HTML document.
- JavaScript can access and update the content, structure and style of the page.



HTML

```
<h1 id="title">Hello</h1>
<p class="msg">Welcome!</p>
```

2 SELECTING ELEMENTS

Method	Description	Example	Returns
getElementById()	Select by id	document.getElementById('title')	Single element
getElementsByName()	Select by class	document.getElementsByName('msg')	HTMLCollection
getElementsByTagName()	Select by tag	document.getElementsByTagName('p')	HTMLCollection
querySelector()	Select first match	document.querySelector('.msg')	Single element
querySelectorAll()	Select all matches	document.querySelectorAll('.msg')	NodeList



Use `querySelector` for a single element.
Use `querySelectorAll` for multiple elements.

3 CHANGING CONTENT

```
element.innerHTML = "New <b>HTML</b>"; // Sets HTML content
element.textContent = "New text"; // Sets text only
element.innerText = "New text"; // Visible text (respects CSS)
```

EXAMPLE

```
const title = document.getElementById("title");
title.innerHTML = "Hi, <span style='color:tomato'>Coding Artist!</span>";
title.textContent = "Hi, Coding Artist!";
```



4 CHANGING STYLES

```
element.style.color = "tomato";
element.style.backgroundColor = "#111";
element.style.fontSize = "20px";

element.classList.add("active");
element.classList.remove("active");
element.classList.toggle("active");
element.classList.contains("active");
```

EXAMPLE

```
const box = document.querySelector('.box');
box.style.color = "white";
box.classList.add("highlight");
```

CLASSLIST METHODS

`add()` → Add a class
`remove()` → Remove a class
`toggle()` → Add or remove class
`contains()` → Check if class exists

TIP

It's better to add/remove classes instead of changing styles directly.

5 CREATING & ADDING ELEMENTS

```
const newEl = document.createElement('p'); // Create element
newEl.textContent = "I am dynamically added!";
```

```
element.appendChild(newEl); // Add at end
element.prepend(newEl); // Add at start
element.insertBefore(newEl, refEl); // Add before reference element
element.after(newEl); // Add after element
element.before(newEl); // Add before element
```

EXAMPLE

```
const list = document.getElementById('list');
const item = document.createElement('li');
item.textContent = "New Item";
list.appendChild(item);
```



`appendChild()` is widely supported. `append()` is simpler and can add multiple nodes or text.

6 REMOVING ELEMENTS

```
element.remove();
parent.removeChild(child); // Old way
```

EXAMPLE

```
const item = document.querySelector('.item');
item.remove();
// or
const parent = document.getElementById('list');
const child = document.querySelector('.item');
parent.removeChild(child);
```

NOTE:

`remove()` is the modern and preferred way.

7 ATTRIBUTES & PROPERTIES

GET / SET ATTRIBUTES

```
element.getAttribute('attr');
element.setAttribute('attr', 'value');
element.removeAttribute('attr');
```

EXAMPLE

```
const link = document.querySelector('a');
link.setAttribute('href', 'https://google.com');
link.removeAttribute('target');
```

COMMON PROPERTIES

```
element.id
element.className
element.value // for input, select, etc.
element.checked // for checkbox, radio
element.disabled // true / false
```



8 EVENT LISTENERS (BASICS)

```
element.addEventListener('event', callback);
```

EXAMPLE

```
const btn = document.querySelector('#btn');
btn.addEventListener('click', function() {
  alert("Button clicked!");
});
```

COMMON EVENTS

`click` → Mouse click
`input` → Typing in input
`change` → Value change
`submit` → Form submit
`mouseover` → Mouse over
`keydown` → Key pressed

9 QUICK MINI EXAMPLES

1. Change Text

```
document.getElementById('title')
.textContent = "New Title";
// New Title
```

2. Change Style

```
document.querySelector('.box')
.style.backgroundColor = "skyblue";
// background changed
```

3. Add Element

```
const p = document.createElement('p');
p.textContent = "Hello DOM!";
document.body.appendChild(p);
// <p>Hello DOM!</p> added
```

4. Remove Element

```
const el = document.querySelector('.temp');
el.remove();
// element removed
```

5. Toggle Class

```
const box = document.querySelector('.box');
box.classList.toggle('active');
// class toggled
```



PRACTICE TIP: Inspect elements using Developer Tools (Right Click → Inspect) and try changing things with JavaScript!

</> Coding Artist </>

codingartistweb.com/shop

EVENTS & EVENT HANDLING



Listen. Respond. Interact. Make your web pages alive! ❤️

1 WHAT IS AN EVENT?

- ✓ An event is something that happens in the browser that JavaScript can respond to.
- ✓ Examples: click, key press, mouse move, page load, etc.

```
<button id="btn">Click Me</button>
<p id="msg"></p>
```

EVENTS CAN COME FROM:

- ✓ User actions
- ✓ Browser actions
- ✓ Page / Element load
- ✓ Form interactions

2 COMMON EVENTS

Event	What it does	Example
click	Mouse click on element	button click
dblclick	Double click on element	image double click
mouseover	Mouse pointer over element	hover over box
mouseout	Mouse pointer out of element	leave the box
mousemove	Mouse moves over element	track mouse movement
keydown	Key is pressed down	press any key
keyup	Key is released	release any key
submit	Form is submitted	form submit
change	Value of input/select changes	select option change
load	Page or resource is loaded	window load



Event names are case-sensitive (use lowercase).
Most events bubble up to parent elements.



3 ADDING EVENT LISTENERS

Syntax

```
element.addEventListener("event", callback);
```

- event → type of event (e.g. "click")
- callback → function to run when event occurs

Example

```
const btn = document.getElementById("btn");
btn.addEventListener("click", function() {
  alert("Button was clicked!");
});
```

Using Arrow Function

```
btn.addEventListener("click", () => {
  alert("Clicked with arrow function!");
});
```

Multiple Listeners

```
btn.addEventListener("click", () => {
  console.log("First listener");
});
btn.addEventListener("click", () => {
  console.log("Second listener");
});
```



You can add multiple listeners for the same event.

4 EVENT OBJECT

When an event occurs, an event object is passed to the callback function.

Example

```
btn.addEventListener("click", (event) => {
  console.log(event.type); // click
  console.log(event.target); // <button id="btn">
  console.log(event.clientX); // mouse X position
  console.log(event.clientY); // mouse Y position
});
```

Useful Properties

```
event.type
event.target
event.currentTarget
event.clientX / clientY
event.key
event.code
```



event.target = the element that triggered the event.
event.currentTarget = the element the listener is attached to.

5 COMMON EXAMPLES

1. Click Event

```
btn.addEventListener("click", () => {
  document.getElementById("msg").textContent = "Button Clicked!";
});
```

2. Input Event

```
const input = document.getElementById("name");
input.addEventListener("input", (e) => {
  console.log("You typed:", e.target.value);
});
```

3. Change Event

```
const select = document.getElementById("fruit");
select.addEventListener("change", (e) => {
  console.log("Selected:", e.target.value);
});
```

4. Keydown Event

```
document.addEventListener("keydown", (e) => {
  console.log("Key pressed:", e.key);
});
```

5. Submit Event

```
const form = document.getElementById("myForm");
form.addEventListener("submit", (e) => {
  e.preventDefault(); // stop form from reloading page
  alert("Form submitted!");
});
```



Use `e.preventDefault()` in forms to stop default behavior (like page reload).

6 THIS KEYWORD IN EVENTS

Inside a regular function, this refers to the element that triggered the event.

With Regular Function

```
btn.addEventListener("click", function() {
  this.style.backgroundColor = "lightgreen";
}); // 'this' is the button
```

With Arrow Function

```
btn.addEventListener("click", () => {
  // 'this' does NOT refer to button here
  console.log(this); // window
});
```

REMEMBER

Use regular function if you need "this" to refer to the element.

7 EVENT LISTENER OPTIONS

You can pass an options object as the third argument.

Options

- once: true → Runs only once
- passive: true → Tells browser not to call `preventDefault()`
- capture: true → Runs in capture phase

Example (runs only once)

```
btn.addEventListener("click", () => {
  alert("I run only once!");
}, { once: true });
```



8 REMOVING EVENT LISTENERS

Use `removeEventListener()` to remove a listener.

Example

```
function handleClick() {
  alert("Button clicked!");
}
btn.addEventListener("click", handleClick);
// Later...
btn.removeEventListener("click", handleClick);
```



You must pass the same function reference to remove the listener.

9 EVENT FLOW (BUBBLING & CAPTURING)

1. Capturing Phase

Event travels from window → down to the target element.



2. Bubbling Phase (Default)

Event travels from the target element → up to window.



Use `{ capture: true }` to listen in capturing phase.



TIP: Events are the bridge between your code & users. Master them to build interactive & dynamic web apps! 🚀

</> Coding Artist </>

codingartistweb.com/shop

MODERN JAVASCRIPT

Destructuring, Spread, Rest & Modules

JS

1 DESTRUCTURING ASSIGNMENT

Extract values from arrays or objects into distinct variables.

Array Destructuring

```
const numbers = [10, 20, 30];
const [a, b, c] = numbers;
// a = 10, b = 20, c = 30
```

Skipping Items

```
const [first, , third] = numbers;
// first = 10, third = 30
```

Object Destructuring

```
const person = {
  name: 'Mitali',
  age: 30,
  city: 'Pune'
};
```

```
const { name, age } = person;
// name = 'Mitali', age = 30
```

Renaming Variables

```
const { name: n, city: location } = person;
// n = 'Mitali', location = 'Pune'
```

WHY USE IT?

- ✓ Clean & concise code
- ✓ Easier to read
- ✓ Extract only what you need



Destructuring works with nested data too!

2 DEFAULT VALUES IN DESTRUCTURING

Array Default

```
const [x = 1, y = 2] = [];
// x = 1, y = 2
```

```
const [a = 10, b = 20] = [5];
// a = 5, b = 20
```

Object Default

```
const { name = 'Guest', age = 0 } = {};
// name = 'Guest', age = 0
```

```
const { city = 'Pune' } = { city: 'Mumbai' };
// city = 'Mumbai'
```



Defaults are used only when the value is undefined.

3 SPREAD OPERATOR (...)

Spread expands an iterable (array, object, string) into individual elements.

Spread with Arrays

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5];
const combined = [...arr1, ...arr2];
// [1, 2, 3, 4, 5]

const copy = [...arr1];
// Shallow copy of arr1
```

Spread with Objects

```
const user = { name: 'Mitali', age: 30 };
const moreInfo = { city: 'Pune', role: 'Dev' };
const merged = { ...user, ...moreInfo };
// { name: 'Mitali', age: 30, city: 'Pune', role: 'Dev' }
```



Spread creates shallow copies for arrays & objects.

4 REST PARAMETERS / REST OPERATOR

Rest collects multiple items into a single array or object.

Rest in Functions

```
function sum(...nums) {
  return nums.reduce((t, n) => t + n, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
console.log(sum(5, 10)); // 15
```

Rest in Destructuring

```
const [first, ...rest] = [10, 20, 30, 40];
// first = 10, rest = [20, 30, 40]

const { name, ...others } = {
  name: 'Mitali',
  age: 30,
  city: 'Pune',
  role: 'Dev'
};
// name = 'Mitali', others = { age: 30, city: 'Pune', role: 'Dev' }
```



Rest must be the last element in array destructuring and last property in object destructuring.

5 COMBINING SPREAD & REST

Example - Function Arguments

```
const nums = [1, 2, 3, 4];
console.log(sum(...nums)); // 10
// Spread array as arguments
```

Example - Return Multiple Values

```
function minMax(...values) {
  return [Math.min(...values), Math.max(...values)];
}

const [min, max] = minMax(5, 1, 9, 2);
// min = 1, max = 9
```



Power combo for flexible functions and clean code!

6 MODULES (ES6 MODULES)

Modules help you split code into separate files and reuse them.

Exporting (math.js)

```
export const PI = 3.14159;
export function add(a, b) {
  return a + b;
}

const multiply = (a, b) => a * b;
export default multiply;
```

Importing (app.js)

```
import { PI, add } from './math.js';
import multiply from './math.js';

console.log(PI); // 3.14159
console.log(add(2, 3)); // 5
console.log(multiply(2, 4)); // 8
```

Use type="module" in HTML script tag:
<script type="module" src="app.js"></script>

7 NAMED EXPORTS vs DEFAULT EXPORT

Named Export

- ✓ Export multiple things
- ✓ Use curly braces {}
- // math.js
- export const PI = 3.14;
- export function add() {}

```
// Named
import { PI, add } from './math.js';
```

Default Export

- ✓ Export only one item per file
- ✓ No curly braces on import
- // math.js
- export default function greet() {}

```
// Default
import greet from './math.js';
```

NOTE

A file can have many named exports but only one default export.

8 DYNAMIC IMPORT (ADVANCED)

Lazy load modules when needed.

```
button.addEventListener('click', async () => {
  const { add } = await import('./math.js');
  console.log(add(10, 20)); // 30
});
```



Great for large apps. Loads modules on demand.

WHY USE IT?

- ✓ Better performance
- ✓ Code splitting
- ✓ Load only what you need

9 QUICK EXAMPLES

1. Swap Two Variables

```
[a, b] = [b, a];
```

2. Clone Array

```
const clone = [...original];
```

3. Merge Objects

```
const newObj = { ...obj1, ...obj2 };
```

4. Collect Rest Items

```
const [x, ...others] = [1, 2, 3, 4];
```

5. Use Module

```
import { add } from './math.js';
```



TIP: Master these modern features to write clean, efficient and professional JavaScript code!

</> Coding Artist </>

codingartistweb.com/shop

ADVANCED JAVASCRIPT

Closures, this & Prototypes

JS

1 CLOSURES

- ✓ A closure is a function that remembers variables from its outer (enclosing) scope even after the outer function has finished executing.

```
function outer(name) {
  let message = 'Hello, ${name}!';
  function inner() {
    console.log(message); // remembers message
  }
  return inner;
}

const greet = outer('Mitali');
greet(); // Hello, Mitali!
```

WHY USE CLOSURES?

- ✓ Data privacy
- ✓ State persistence
- ✓ Function factories
- ✓ Callbacks & Event handlers



Inner function has access to variables, parameters and functions of its outer function.

2 PRACTICAL CLOSURE EXAMPLE

Counter using Closure

```
function createCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

Private Variables

```
function bankAccount(initialBalance) {
  let balance = initialBalance; // private
  return {
    deposit(amount) { balance += amount; },
    withdraw(amount) {
      if (amount > balance) console.log("Insufficient");
      else balance -= amount;
    },
    getBalance() { return balance; }
  };
}

const acc = bankAccount(1000);
acc.deposit(500);
console.log(acc.getBalance()); // 1500
```



Closures let you create private data and maintain state. Very useful in real-world applications.

3 THE this KEYWORD

- ✓ this refers to the object that is executing the current function.
✓ Its value depends on how the function is called.

Context	What this refers to	Example
Global	Window object (browser)	console.log(this);
Object Method	Object before the dot	obj.method();
Function (strict mode)	undefined	"use strict"; function f() {}
Constructor	New instance	new Person();
Event Handler (DOM)	Element that triggered	btn.addEventListener('click', function() { console.log(this); // button });
Arrow Function	Inherited from surrounding scope	setTimeout(() => { console.log(this); }, 100);

EXAMPLE

```
const user = {
  name: "Coding Artist",
  sayHi: function() {
    console.log("Hi, I am ${this.name}");
  }
};

user.sayHi(); // Hi, I am Coding Artist
```

NOTES

- ✓ Arrow functions don't have their own this.
- ✓ Use bind, call, apply to control this.



4 CALL, APPLY & BIND

call()

```
const person = {
  name: "Mitali"
};

function greet(city) {
  console.log("Hi, I am ${this.name}");
  console.log("from ${city}");
}

greet.call(person, "Pune");
// Hi Mitali from Pune
```

apply()

```
const person = {
  name: "Mitali"
};

function greet(city, lang) {
  console.log("Hi, I am ${this.name}");
  console.log("from ${city}, ${lang}");
}

greet.apply(person, ["Pune", "JS"]);
// Hi Mitali from Pune, JS
```

bind()

```
const person = {
  name: "Mitali"
};

function greet(lang) {
  console.log("Hi, I am ${this.name}");
  console.log("from ${lang}");
}

const hi = greet.bind(person, "JS");
hi(); // Hi Mitali, JS
```



call() and apply() call the function immediately. bind() returns a new function with this bound.



5 PROTOTYPES (THE BASICS)

- ✓ Every JavaScript object has a hidden property [[Prototype]].
✓ Objects inherit properties and methods from their prototype.

Using Object Literal

```
const user = { name: "Mitali" };
console.log(user.toString()); // inherited
console.log(user.hasOwnProperty("name")); // true
console.log(user.hasOwnProperty("toString")); // false
```

Prototype Chain

```
user.__proto__ === Object.prototype; // true
Object.prototype.__proto__ === null; // true
// Chain: user → Object.prototype → null
```



Prototypes enable inheritance and code reusability.

6 PROTOTYPES & CONSTRUCTOR FUNCTIONS

Constructor Function

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

Person.prototype.greet = function() {
  console.log("Hi, I am ${this.name}");
};

const p1 = new Person("Mitali", 30);
p1.greet(); // Hi, I am Mitali
```

Prototype Inheritance

```
function Student(name, age, course) {
  Person.call(this, name, age);
  this.course = course;
}

Student.prototype = Object.create(Person.prototype);
Student.prototype.constructor = Student;
Student.prototype.study = function() {
  console.log(`${this.name} is studying ${this.course}`);
};

const s1 = new Student("Mitali", 30, "JavaScript");
s1.greet(); // Hi, I am Mitali
s1.study(); // Mitali is studying JavaScript
```



Prototypes are the foundation of JavaScript inheritance.

7 QUICK EXAMPLES

Closure

```
function makeAdder(x) {
  return function(y) {
    return x + y;
  };
}

const add5 = makeAdder(5);
console.log(add5(3)); // 8
```

this in Method

```
const obj = {
  value: 42,
  show() { console.log(this.value); }
};

obj.show(); // 42
```

Arrow this

```
const obj = {
  value: 42,
  show: () => console.log(this.value)
};

obj.show(); // undefined (no this)
```

Prototype Method

```
Array.prototype.last = function() {
  return this[this.length - 1];
};

console.log([1, 2, 3].last()); // 3
```

REMEMBER

- ✓ Understand scopes
- ✓ Know how this works
- ✓ Master prototypes to write powerful JS!



TIP: Advanced concepts = Strong JavaScript foundation = Better Code & Problem Solving!

</> Coding Artist </>

codingartistweb.com/shop

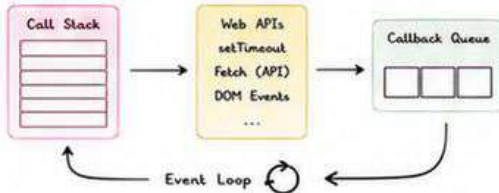
ASYNCHRONOUS JAVASCRIPT

Callbacks, Promises & Async/Await

JS

1 WHAT IS ASYNCHRONOUS JAVASCRIPT?

- JS is single-threaded, but it can perform non-blocking operations using the Event Loop.
- Helps handle tasks like API calls, timers, file reading, etc.



When the call stack is empty, the event loop moves callbacks from the queue to the stack.

2 CALLBACKS

- A callback is a function passed as an argument to another function.
- It runs after the async operation is completed.

EXAMPLE

```
function fetchData(callback) {
  setTimeout(() => {
    const data = { name: "Mitali", city: "Pune" };
    callback(data); // callback after 2s
  }, 2000);
}

fetchData((data) => {
  console.log("Data received:", data);
});
```

PROS & CONS

- ✓ Simple to understand
- ✓ Widely supported
- ✗ Callback hell (nested callbacks are hard to manage)



Callbacks can lead to "Callback Hell" when nested multiple times. Promises & Async/Await solve this problem.

3 PROMISES

- A Promise is an object that represents the eventual completion (or failure) of an async operation.
- It has 3 states: Pending, Fulfilled, Rejected.
- Once settled, a promise can't change its state.

EXAMPLE

```
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true;
    if (success) {
      resolve({ name: "Mitali", city: "Pune" });
    } else {
      reject("Something went wrong!");
    }
  }, 2000);
});
```

Using then() & catch()

```
fetchData
  .then((data) => {
    console.log("Success:", data);
  })
  .catch((error) => {
    console.log("Error:", error);
  })
  .finally(() => {
    console.log("Done!");
  });
```



then() runs on success, catch() on failure, finally() always runs.

4 ASYNC / AWAIT

- Async/Await is syntactic sugar over Promises.
- Makes async code look and behave like synchronous code.
- It's easier to read and write.

EXAMPLE

```
async function getData() {
  try {
    const data = await fetchData(); // wait for promise
    console.log("Success:", data);
  } catch (error) {
    console.log("Error:", error);
  } finally {
    console.log("Done!");
  }
}

getData();
```

KEY POINTS

- ✓ async before function always returns a Promise.
- ✓ await can only be used inside async functions.
- ✓ try...catch handles errors elegantly.



Async/Await makes code clean, readable & easy to debug.

5 COMPARISON

Feature	Callbacks	Promises	Async/Await
Readability	Hard (Callback Hell)	Better	Best
Error Handling	Hard (Nested)	catch()	try...catch
Chaining	Nested functions	then() chaining	Sequential code
Return Value	No direct return	Returns a Promise	Returns a Promise
Use Case	Old projects	Better control	Modern JS (ES8+)

6 PRACTICAL EXAMPLE - FETCH API

Using Promises

```
fetch("https://api.example.com/users")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log("Error:", err));
```

Using Async/Await

```
async function loadUsers() {
  try {
    const res = await fetch("https://api.example.com/users");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log("Error:", err);
  }
}

loadUsers();
```



Async/Await is the preferred way for modern JavaScript. It improves readability and maintainability.

7 USEFUL METHODS

Promise Methods

Promise.resolve(value) → Returns resolved promise
 Promise.reject(error) → Returns rejected promise
 Promise.all([p1, p2]) → All must succeed
 Promise.allSettled([p1, p2]) → Waits for all (success or fail)
 Promise.race([p1, p2]) → First settled promise
 Promise.any([p1, p2]) → First fulfilled promise

Async/Await Tips

- ✓ Use await only when necessary
- ✓ Handle errors with try...catch
- ✓ Avoid blocking the main thread
- ✓ Keep async functions small & focused



Great for running multiple async tasks in parallel.

8 QUICK EXAMPLES

1. setTimeout with Promise

```
function delay(ms) {
  return new Promise(res =>
    setTimeout(res, ms)
  );
}

delay(1000).then(() =>
  console.log("1s passed!")
);
```

2. Wait for Multiple Tasks

```
const p1 = fetch("/api/a");
const p2 = fetch("/api/b");
Promise.all([p1, p2])
  .then(responses =>
    console.log("All done!")
  );
```

3. Async/Await with Loop

```
async function loadAll(ids) {
  for (let id of ids) {
    const res = await fetch(`/api/${id}`);
    const data = await res.json();
    console.log(data);
  }
}

loadAll([1, 2, 3]);
```



TIP: Understand the Event Loop, master Promises, and write clean Async/Await code. You'll build faster, better & smarter JavaScript applications!

</> Coding Artist </>

codingartistweb.com/shop